

indexrelname	idx_scan	idx_tup_read	idx_tup_fetch
posts_pkey	5	5	5
idx_posts_date	1	500000	500000
idx_posts_author	51	8534	0

This tells us that `idx_posts_date` has been used 1 time, providing 500000 tuples, while `idx_posts_author` has been used 51 times and also provided far fewer tuples (only 8534), meaning it is very effective.

After observing this trend as time goes by, if the `idx_posts_date` is seldom used, you can safely drop it.

As a last example, let's consider a poorly written query and the problem it implies: assume we want to extract all the authors that created posts on a certain date and that received, on the same date, a specific number of likes. A bad query could be the following:

```
SELECT u.username
FROM users u JOIN posts p ON p.author = u.pk WHERE p.created_on = CURRENT_
DATE - 5
AND u.pk IN ( SELECT pp.author FROM posts pp WHERE likes = 5 and
p.created_on = created_on );
```

Clearly, there is no need for the subquery on the posts table, but this is a specific example to demonstrate why it is important to understand how to write good queries.

At a glance, the preceding query runs fast enough; it takes 190 milliseconds to execute, as demonstrated by `EXPLAIN ANALYZE`:

```
forumdb=> EXPLAIN ANALYZE SELECT u.username
FROM users u JOIN posts p ON p.author = u.pk WHERE p.created_on = CURRENT_
DATE - 5
AND u.pk IN ( SELECT pp.author FROM posts pp WHERE likes = 5 and
p.created_on = created_on );
```

QUERY PLAN

```
-----
Hash Join (cost=33.93..92.04 rows=498 width=15) (actual
time=1.351..189.308 rows=100 loops=1)
  Hash Cond: (p.author = u.pk)
```

```

Join Filter: (SubPlan 1)
Rows Removed by Join Filter: 900
-> Index Scan using idx_posts_date on posts p (cost=0.43..55.92
rows=996 width=12) (actual time=0.029..0.254 rows=1000 loops=1)
    Index Cond: (created_on = (CURRENT_DATE - 5))
-> Hash (cost=21.00..21.00 rows=1000 width=19) (actual
time=0.345..0.345 rows=1000 loops=1)
    Buckets: 1024 Batches: 1 Memory Usage: 59kB
-> Seq Scan on users u (cost=0.00..21.00 rows=1000 width=19)
(actual time=0.006..0.151 rows=1000 loops=1)
SubPlan 1
-> Index Scan using idx_posts_date on posts pp (cost=0.42..58.49
rows=99 width=4) (actual time=0.005..0.176 rows=95 loops=1000)
    Index Cond: (created_on = p.created_on)
    Filter: (likes = 5)
    Rows Removed by Filter: 855
Planning Time: 0.393 ms
Execution Time: 189.439 ms

```

However, looking at the output, you can see that the **Index Scan** node using `idx_posts_date` on `pp posts` has a loops counter set to `1000`: this means that this node is executed 1,000 times in the query. The node is the subquery, and that is how SQL can achieve the result: every time a tuple from the outer query is found, the inner query must be run and the results have to be joined. One possible option is to rewrite the query avoiding PostgreSQL having to do the looping, for example, by use of a subquery with a `GROUP BY` expression:

```

forumdb=> EXPLAIN ANALYZE
WITH likes AS (
    SELECT pp.author, pp.created_on FROM posts pp
    WHERE likes = 5
    GROUP BY pp.author, pp.created_on
)
SELECT u.username
FROM users u JOIN posts p ON p.author = u.pk
JOIN likes l ON l.created_on = p.created_on AND l.author = u.pk
WHERE p.created_on = CURRENT_DATE - 5;

```

```

-----
-----
-----
Hash Join (cost=89.47..149.68 rows=98 width=15) (actual time=0.581..0.896
rows=100 loops=1)
  Hash Cond: (p.author = u.pk)
    -> Index Scan using idx_posts_date on posts p (cost=0.43..55.92
rows=996 width=12) (actual time=0.012..0.167 rows=1000 loops=1)
      Index Cond: (created_on = (CURRENT_DATE - 5))
    -> Hash (cost=87.82..87.82 rows=98 width=31) (actual time=0.561..0.562
rows=100 loops=1)
      Buckets: 1024 Batches: 1 Memory Usage: 15kB
    -> Hash Join (cost=62.09..87.82 rows=98 width=31) (actual
time=0.305..0.540 rows=100 loops=1)
      Hash Cond: (u.pk = l.author)
        -> Seq Scan on users u (cost=0.00..21.00 rows=1000
width=19) (actual time=0.005..0.094 rows=1000 loops=1)
        -> Hash (cost=60.86..60.86 rows=98 width=12) (actual
time=0.294..0.294 rows=100 loops=1)
          Buckets: 1024 Batches: 1 Memory Usage: 13kB
          -> Subquery Scan on l (cost=58.90..60.86 rows=98
width=12) (actual time=0.243..0.276 rows=100 loops=1)
            -> HashAggregate (cost=58.90..59.88 rows=98
width=12) (actual time=0.242..0.260 rows=100 loops=1)
              Group Key: pp.author, pp.created_on
              Batches: 1 Memory Usage: 24kB
              -> Index Scan using idx_posts_date on
posts pp (cost=0.43..58.41 rows=98 width=12) (actual time=0.009..0.208
row
s=100 loops=1)
                Index Cond: (created_on = (CURRENT_
DATE - 5))
              Filter: (likes = 5)
              Rows Removed by Filter: 900
Planning Time: 0.455 ms
Execution Time: 0.954 ms

```

This reports back the time at less than a millisecond, and most notably, does not make PostgreSQL loop over the same subquery.

Clearly, writing the query without any subquery at all is the most efficient way to let PostgreSQL understand what it has to do:

```

forumdb=> EXPLAIN ANALYZE
SELECT u.username
FROM users u JOIN posts p ON p.author = u.pk
WHERE p.created_on = CURRENT_DATE - 5
AND p.likes = 5;

```

QUERY PLAN

```

-----
Hash Join (cost=33.93..92.17 rows=98 width=15) (actual time=0.413..0.638
rows=100 loops=1)
  Hash Cond: (p.author = u.pk)
    -> Index Scan using idx_posts_date on posts p (cost=0.43..58.41
rows=98 width=4) (actual time=0.015..0.212 rows=100 loops=1)
      Index Cond: (created_on = (CURRENT_DATE - 5))
      Filter: (likes = 5)
      Rows Removed by Filter: 900
    -> Hash (cost=21.00..21.00 rows=1000 width=19) (actual
time=0.393..0.394 rows=1000 loops=1)
      Buckets: 1024 Batches: 1 Memory Usage: 59kB
      -> Seq Scan on users u (cost=0.00..21.00 rows=1000 width=19)
(actual time=0.007..0.173 rows=1000 loops=1)
Planning Time: 0.263 ms
Execution Time: 0.666 ms

```

The preceding simple example demonstrates why it is important to understand the output of EXPLAIN and EXPLAIN ANALYZE in order to better understand if the query is missing an index or needs a rewrite to let PostgreSQL do its job best.

ANALYZE and how to update statistics

PostgreSQL exploits a statistical approach to evaluate different execution plans. This means that PostgreSQL does not know how many tuples there are in a table, but has a good approximation that allows the planner to compute the cost of the execution plan.

Statistics are not only related to the quantity (how many tuples) but also to the quality of the underlying data – for example, how many distinct values there are, which values are more frequent in a column, and so on. Thanks to the combination of all of this data, PostgreSQL is able to make a performant decision.

There are times, however, when the quality of the statistical data is not good enough for PostgreSQL to choose the best plan, a problem commonly known as “out-of-date statistics.” In fact, statistics are not updated in real time; rather, PostgreSQL keeps track of what is ongoing in every table in every database and summarizes the number of new tuples, updated ones, and deleted ones, as well as the quality of their data. It could happen that the statistics are not updated frequently enough (or not at all) for different reasons that we are going to explain later in this chapter, so the DBA should always have a way to force PostgreSQL to start from scratch and “rebuild” the statistics.

The command that does this is `ANALYZE`.



The `ANALYZE` command has nothing to do with the `ANALYZE` option of the `EXPLAIN` command; rather, it is similar to the option with the same name used with the `VACUUM` command, as explained in *Chapter 11*.

`ANALYZE` accepts a table (and, optionally, a list of columns) and builds all the statistics for the specified table (or the specified columns only).

As important as it is to keep the statistics up to date, running `ANALYZE` manually is not a good habit, and it is for that reason that the auto-analyze daemon is in charge of periodically updating the statistics when enough changes happen on a table.

The syntax for the `ANALYZE` command is the following:

```
ANALYZE [ ( option [, ...] ) ] [ table_and_columns [, ...] ]
```

Essentially, it can be launched against a single table as follows:

```
forumdb=> \timing
forumdb=> ANALYZE posts;
ANALYZE
Time: 252.771 ms
```



You can inspect the times required to execute commands and queries with the `\timing psql` special command, which will print a summary of the time elapsed after every statement. This is not a solid way to measure performance, only to get an idea of how much time a task is taking.

`ANALYZE` does not support a lot of options, mainly `VERBOSE` to display a verbose output of what `ANALYZE` is doing, and `SKIP_LOCKED`, which makes `ANALYZE` skip a table if it cannot acquire the appropriate locks because there are other ongoing operations that have already acquired an incompatible lock.

Where does PostgreSQL store the statistics that `ANALYZE` collects? The `pg_stats` special catalog contains all the statistics used by the planner to determine the values and constraints to examine the attributes. For example, let's see what PostgreSQL knows about the `author` column of the `posts` table, and in particular, how many distinct values there are:

```
forumdb=> SELECT n_distinct
           FROM pg_stats
           WHERE attname = 'author' AND tablename = 'posts';
 n_distinct
-----
        1000
(1 row)
```

PostgreSQL knows that we have 1,000 different authors that have posted at least one post in our example database, as demonstrated by the `EXPLAIN ANALYZE` command in the previous section.

One bit of important information you can find in the `pg_stats` catalog is the most common values, correlated by the frequency that these values appear. Extracting this information requires a little more attention since both values and frequencies are stored as arrays, so the following query provides the most common values and frequency for the `author` column:

```
forumdb=> select most_common_vals, most_common_freqs from pg_stats where
tablename = 'posts' and attname = 'category';
-[ RECORD 1 ]-----+-----
--
most_common_vals  | {3,4,1,5,2}
most_common_freqs | {0.20566666,0.20163333,0.19843334,0.19743334,0.1968333
3}
```

The output of the query means that the category with primary key 3 appears with a frequency of 0.2056, the category with primary key 4 appears with a frequency of 0.2016, and so on. The frequency translates into the number of tuples by multiplying the tuples contained in the table (500,000) by the frequency. Therefore, the category with primary key 3 appears in $500000 \times 0.2056 = 102,800$ tuples of the `posts` table. This can be easily checked with the following query:

```

forumdb=> SELECT count(*), category
FROM posts
GROUP BY category
ORDER BY 2;
count | category
-----+-----
100000 |      1
100000 |      2
100000 |      3
100000 |      4
100000 |      5
(5 rows)

```

The result extracted from `pg_stats` is not the same as the `SELECT count(*)` query because `pg_stats` is not meant to provide an absolute and accurate result, rather an order of magnitude.

There is other information in `pg_stats`, such as the number of NULL values for a column, the number of distinct values, and so on.

In conclusion, PostgreSQL keeps track of the statistics of every column in every table; the statistics are updated by `ANALYZE` or the auto-analyze daemon so that the planner can always be trusted to have a good approximation of the quantity and quality of the data that is stored in a table.

Auto-explain

Auto-explain is an extension that helps the DBA get an idea of slow queries and their execution plan. Essentially, auto-explain triggers when a running query is slower than a specified threshold, and then dumps the execution plan of the query in the PostgreSQL logs (refer to *Chapter 14, Logging and Auditing*, for more detail).



Note: the Docker image for this chapter comes with auto-explain and log machinery pre-configured.

In this way, the DBA can get an insight into slow queries and their execution plans without having to re-execute these queries. Thanks to this, the DBA can inspect the execution plans and decide if and where to apply indexes or perform a deeper analysis.

The auto-explain module is configured via a set of `auto_explain` parameter options that can be inserted in the PostgreSQL configuration (the `postgresql.conf` file), but you need to remember that in order to activate the module, you need to restart the cluster.

The auto-explain module can do pretty much the same things that a manual `EXPLAIN` command can do, including `EXPLAIN ANALYZE`, but it has to be properly configured.

All the settings for auto-explain are named in the namespace `auto_explain`, therefore any parameter has a prefix that starts with `auto_explain.`; the main settings are the following ones:

- `auto_explain.log_min_duration` is the threshold of time a statement must take before it is logged. Any statement requiring more time than this setting will appear in the cluster logs with its `EXPLAIN` output.
- `auto_explain.log_format` and `auto_explain.log_level` control the format of the output, in terms of text, JSON, YAML, XML and the level at which such output will be logged (e.g., `INFO`).
- `auto_explain.log_verbose`, if turned on, provides more verbose information in the output.
- `auto_explain.sample_rate` is a value between 0 and 1 indicating the sampling rate of a session. For example, 0.5 means that one statement out of two will be logged.
- `auto_explain.log_nested_statements` is a boolean value that determines whether “inner” statements have to be logged on their own. For example, if this option is turned on when logging a function call, statements that also happen inside the function will be logged and explained.
- `auto_explain.log_analyze` is a boolean value that indicates whether the logged statement must also report `EXPLAIN ANALYZE` values, mainly the actual timing. Be aware that taking per-node timing information can be resource-demanding and thus can slow down the whole query. When this parameter is turned on, other settings can be turned on to provide the same information that `EXPLAIN ANALYZE` does:
- `auto_explain.log_buffers`, when turned on, provides information about the buffer’s utilization.
- `auto_explain.log_wal`, when turned on, provides information about the WAL produced by a query.

- `auto_explain.log_timing`, when turned on, provides per-node timing information.
- `auto_explain.log_triggers`, when turned on, provides information about trigger executions within a statement.
- `auto_explain.log_settings`, when turned on, reports settings different from the cluster-wide configuration.

In order to install and configure the module, let's start simple and add the following two settings to the cluster configuration in `postgresql.conf`:

```
session_preload_libraries = 'auto_explain'
auto_explain.log_min_duration = '100ms'
```

The first line tells PostgreSQL to load the library related to the auto-explain module, while the second instruments the module to trigger whenever a query takes longer than 100 milliseconds to conclude. Of course, you can raise the query duration or lower it, depending on your needs.

With that configuration in place, it is now possible to execute quite a long query, as follows (assuming you have dropped/disabled the indexes created in the previous section):

```
forumdb=> \timing
forumdb=> SELECT count(*)
FROM posts p
JOIN users u ON u.pk = p.author
WHERE u.username = 'fluca1978'
AND daterange( CURRENT_DATE - 20, CURRENT_DATE ) @> p.created_on::date;
count
-----
      20
(1 row)

Time: 142.629 ms
```

The query took 142 milliseconds, enough time to trigger our auto-explain, and in fact, in the PostgreSQL logs, you can see the following:

```
$ tail /postgres/16/data/log/postgresql.log
INFO:  duration: 139.933 ms  plan:
Query Text: SELECT count(*)
FROM forum.posts p
```

```

JOIN forum.users u ON u.pk = p.author
WHERE u.username = 'fluca1978'
AND daterange( CURRENT_DATE - 20, CURRENT_DATE ) @> p.created_on::date;
Aggregate (cost=13803.59..13803.60 rows=1 width=8)
Output: count(*)
-> Gather (cost=1008.30..13803.58 rows=2 width=0)
    Workers Planned: 2
    -> Hash Join (cost=8.30..12803.38 rows=1 width=0)
        Inner Unique: true
        Hash Cond: (p.author = u.pk)
        -> Parallel Seq Scan on forum.posts p (cost=0.00..12792.33
            rows=1042 width=4)
...

```

That is exactly the output a normal EXPLAIN command would have produced for the same query.

The beauty of this approach is that you don't have to worry about or remember to execute EXPLAIN on queries or collected queries; you simply have to inspect the logs to find out the execution plan of slow queries. Once you have fixed queries such as the preceding, by creating indexes, for example, you can raise the threshold of auto-explain to catch slower queries and iterate the process again.

In order to demonstrate the difference in the output of running the query with `auto_explain.log_analyze` turned on, the following is the output produced for the very same query:

```

INFO: duration: 139.730 ms plan:
Query Text: SELECT count(*)
FROM forum.posts p
JOIN forum.users u ON u.pk = p.author
WHERE u.username = 'fluca1978'
AND daterange( CURRENT_DATE - 20, CURRENT_DATE ) @> p.created_on::date;
Aggregate (cost=13803.59..13803.60 rows=1 width=8) (actual
time=138.110..139.720 rows=1 loops=1)
Output: count(*)
Buffers: shared hit=487 read=7264
-> Gather (cost=1008.30..13803.58 rows=2 width=0) (actual
time=0.397..139.703 rows=20 loops=1)
    Workers Planned: 2

```